Operating Systems

2-Hour Lecture Notes: Inter-Process Communication and Threads & Concurrency

Reference: Operating System Concepts — Abraham Silberschatz, Peter B. Galvin & Greg Gagne, Addison-Wesley, 2021

Lecture Scope

Part I — IPC: IPC overview, shared memory, message passing, sockets, and RPC.
Part II — Threads & Concurrency: threads overview, processes vs. threads, multiple simultaneous computations, and Amdahl's Law.
Purpose: Detailed notes for lecture understanding, revision, and exam preparation.

Suggested 2-hour flow: IPC concepts and mechanisms (≈60 minutes) → Threads and concurrency (≈50 minutes) → Amdahl's Law and revision (≈10 minutes).

1. Learning Outcomes

After studying these notes, students should be able to:

- Define inter-process communication (IPC) and explain why processes need to communicate.
- Differentiate shared-memory and message-passing IPC models.
- Explain the roles of producer and consumer processes in shared-memory communication.
- Describe direct and indirect message passing, blocking and non-blocking communication, and buffering.
- Explain sockets as an IPC/network communication endpoint and distinguish local communication from networked communication.
- Explain Remote Procedure Call (RPC) and the basic client–server request/response sequence.
- Define a thread and explain the relationship among processes, threads, and process resources.
- Compare single-threaded and multithreaded processes.
- Explain why multithreading is useful for responsiveness, resource sharing, economy, and scalability.
- Explain concurrent execution and distinguish concurrency from parallelism.
- Apply Amdahl's Law to estimate speedup from parallelizing part of a program.

2. Inter-Process Communication (IPC)

2.1 What is IPC?

Inter-Process Communication (IPC) refers to the operating-system mechanisms that allow processes to exchange data and coordinate their activities. A process normally has its own address space, so one process cannot simply read or write another process's memory. The OS therefore provides controlled IPC mechanisms.

> ***Why is IPC needed?***
>
> Processes communicate to exchange information, coordinate execution, share results, synchronize access to resources, and implement client–server or producer–consumer systems. Examples include a web browser communicating with a server process, a shell communicating with commands, and one application passing data to another.

2.2 Two Major IPC Models

Model	Basic idea	Typical characteristics
Shared Memory	Processes communicate by accessing a common memory region.	Very fast after setup; requires synchronization to avoid race conditions.
Message Passing	Processes exchange messages through OS-supported send/receive operations.	Simpler isolation; useful across machines; communication has message-m

3. IPC Using Shared Memory

3.1 Concept

In the **shared-memory model**, the operating system establishes a region of memory that can be accessed by cooperating processes. Once the shared region is established and mapped into their address spaces, processes can communicate by reading and writing data in that region.

- The OS is involved in setting up and managing the shared region.
- After setup, data exchange can be very fast because processes access memory directly.
- Processes must coordinate access to shared data.
- Synchronization tools such as mutexes, semaphores, or monitors may be needed.

• Incorrect synchronization can produce race conditions and inconsistent data.

3.2 Producer–Consumer Example

A common IPC example is the **producer–consumer problem**. A producer generates data and places it into a shared buffer. A consumer removes data from the buffer and processes it.

• **Producer:** creates an item and inserts it into the shared buffer.

• **Consumer:** removes an item from the buffer and uses it.

• **Buffer:** shared data structure between producer and consumer.

• **Synchronization:** prevents the producer from overwriting unavailable space and prevents the consumer from reading an empty buffer.

> ***Important exam point***
>
> Shared memory provides the communication area; it does **not** automatically solve synchronization. When multiple processes access shared data concurrently, synchronization is required to maintain consistency.

3.3 Advantages and Limitations

Advantages	Limitations
Fast data exchange after shared memory is established.	Synchronization must be designed carefully.
Efficient for large amounts of data.	Race conditions may occur.
Processes can exchange data without repeatedly copying messages through the kernel.	More complex programming than simple send/receive communication.
Suitable for producer–consumer and high-throughput systems.	Usually intended for processes on the same system; distributed systems requi

4. IPC Using Message Passing

4.1 Concept

In the **message-passing model**, processes communicate by exchanging messages through OS-provided communication facilities. A process uses operations conceptually similar to **send(message)** and **receive(message)**.

• Processes do not need to share the same address space.

• The OS manages communication channels and message delivery.

• Message passing is especially useful when processes are isolated or execute on different computers.

• It is commonly used in client–server and distributed systems.

4.2 Direct Communication

With **direct communication**, the sender and receiver explicitly identify each other. A conceptual form is **send(P, message)** and **receive(Q, message)**, where P and Q identify processes.

4.3 Indirect Communication

With **indirect communication**, messages are placed in and removed from a communication structure such as a **mailbox** or **port**. The sender sends to the mailbox and a receiver obtains messages from it.

4.4 Blocking and Non-Blocking Communication

Operation	Meaning
Blocking send	The sender may wait until the communication operation satisfies the required condition.

Operation	Meaning
Non-blocking send	The sender continues execution after initiating the send.
Blocking receive	The receiver waits until an appropriate message becomes available.
Non-blocking receive	The receiver continues even if no message is currently available, depending on the interface.

4.5 Buffering

Messages may be stored temporarily in a communication buffer. Buffer capacity affects how independently sender and receiver can execute.

- **Zero capacity:** no messages are queued; sender and receiver may need to synchronize directly.
- **Bounded capacity:** a finite number of messages can wait in the queue.
- **Unbounded capacity:** conceptually, messages can wait without a fixed finite limit.

> ***Shared Memory vs Message Passing***
>
> **Shared memory:** processes communicate by reading/writing a common memory region and must synchronize access. **Message passing:** processes communicate through send/receive operations, providing stronger separation between address spaces.

5. Sockets and Remote Procedure Call (RPC)

5.1 Sockets

A **socket** is an endpoint for communication. In operating systems and networked applications, sockets provide a programming interface through which processes can communicate, including communication between processes on different computers.

- A socket endpoint is associated with an address and communication protocol.
- TCP sockets provide reliable, connection-oriented byte-stream communication.
- UDP sockets provide connectionless datagram communication with lower protocol overhead but without TCP's reliability guarantees.
- Sockets are fundamental to network applications such as web services, chat applications, and database clients.

5.2 Basic Client–Server Socket Flow

- **Server:** creates a socket → binds it to an address → listens for connections → accepts a client → exchanges data.
- **Client:** creates a socket → connects to the server → sends/receives data → closes the connection.
- The exact API calls vary by operating system and programming language, but the conceptual sequence remains similar.

5.3 Remote Procedure Call (RPC)

Remote Procedure Call (RPC) allows a program to invoke a procedure or function on another process or machine using a procedure-call style abstraction. The programmer can think in terms of calling a function, while the underlying RPC mechanism handles communication.

- **Client:** calls a procedure as though it were local.
- **Client stub:** packages (marshals) arguments into a request.
- **Communication system:** sends the request to the remote system.
- **Server stub:** receives and unmarshals the request.
- **Server procedure:** executes the requested operation.

• **Reply:** result is returned through the reverse process.

RPC vs Socket

A socket is a lower-level communication endpoint/interface. RPC provides a higher-level abstraction in which communication is presented as a remote procedure invocation. An RPC system may use sockets or another transport mechanism underneath.

6. Threads and Concurrency

6.1 Thread Overview

A **thread** is a basic unit of CPU utilization within a process. A multithreaded process contains multiple threads that share the process's resources while each thread maintains its own execution state.

- Each thread has its own program counter and register set.
- Each thread has its own stack for function calls and local execution state.
- Threads belonging to the same process share code, data, and operating-system resources such as open files.
- The process provides the resource container; threads provide units of execution.

6.2 Single-Threaded vs Multithreaded Process

Single-threaded	Multithreaded
One execution flow.	Multiple execution flows within one process.
If the thread blocks, the process may appear unresponsive.	Another thread may continue while one thread waits.
Simpler programming model.	Can improve responsiveness and throughput but introduces synchronization c
Limited exploitation of multiple CPU cores.	Can execute threads in parallel on multicore processors when scheduled on d

7. Processes vs. Threads

Aspect	Process	Thread
Address space	Own address space	Shares address space with threads of the process
Resources	Own process resources	Shares many process resources
Execution state	Contains one or more execution contexts	Has its own PC, registers, and stack
Creation/context switch	Generally more expensive	Generally less expensive
Communication	Needs IPC for protected address spaces	Can communicate through shared process memory, requiring synchronizatio
Failure isolation	Stronger isolation between processes	Threads share resources; one thread's memory error can affect the process

Key distinction

A **process** is primarily a resource and protection container, while a **thread** is an execution unit inside a process. Multiple threads of one process share the process's address space and resources.

8. Benefits and Challenges of Multithreading

8.1 Benefits

- **Responsiveness:** one part of an application can continue while another thread waits for I/O.
- **Resource sharing:** threads naturally share the process's code and data.
- **Economy:** creating and switching threads is typically less costly than creating and switching processes.
- **Scalability:** threads can exploit multiple processor cores.
- **Throughput:** independent work can overlap computation and I/O.

8.2 Challenges

- Race conditions when multiple threads access shared data.

- Deadlocks caused by improper synchronization.
- Starvation or unfair resource allocation.
- Difficult debugging because execution order can vary.
- Synchronization overhead can reduce performance.
- Incorrect shared-state design can produce nondeterministic results.

9. Multiple Simultaneous Computations

Modern systems may execute many computations apparently or actually at the same time. The important distinction is between **concurrency** and **parallelism**.

9.1 Concurrency

Concurrency means multiple tasks are in progress during overlapping time periods. On a single CPU core, the OS can create the appearance of simultaneous execution by rapidly switching among tasks.

9.2 Parallelism

Parallelism means two or more computations actually execute simultaneously on different processing resources, such as multiple CPU cores.

Concept	Meaning	Example
Concurrency	Multiple tasks make progress over overlapping time.	One CPU core alternates among threads.
Parallelism	Multiple tasks execute at the same instant.	Two threads run on two CPU cores.

10. Amdahl's Law

10.1 Purpose

Amdahl's Law estimates the theoretical speedup of a program when only part of the program can be improved or parallelized. It demonstrates an important limitation: the portion that cannot be improved places an upper bound on total speedup.

10.2 Formula

Amdahl's Law Formula

Speedup(N) = 1 / [(1 − P) + (P / N)]

Where **P** = fraction of the program that can be parallelized, and **N** = number of processors/cores used for that parallel portion.

10.3 Worked Example

Suppose **80%** of a program can be parallelized, so P = 0.80, and the parallel part runs on **4 cores**.

Speedup = 1 / [(1 − 0.80) + (0.80 / 4)] = 1 / [0.20 + 0.20] = **2.5**.

Therefore, even though four cores are used for the parallelizable part, the overall theoretical speedup is only **2.5×** because 20% of the program remains serial.

10.4 Maximum Speedup

As N approaches infinity, P/N approaches zero. Therefore:

Maximum theoretical speedup

Speedup(max) = 1 / (1 − P)

For P = 0.80, maximum speedup = 1 / 0.20 = **5×**. No matter how many additional processors are added, the serial 20% prevents the program from achieving unlimited speedup.

10.5 Important Interpretation

- Parallel hardware alone does not guarantee proportional performance improvement.
- The serial portion of a program becomes increasingly important as processor count increases.
- Amdahl's Law is a theoretical model; real systems can perform worse because of synchronization, communication, scheduling, memory contention, and other overheads.
- The law is useful for deciding whether further parallelization is likely to provide meaningful performance benefits.

11. Integrated Real-World Scenarios

Scenario 1 — Web Browser

A browser may use multiple processes and multiple threads. Separate processes can provide isolation, while threads within a process can handle UI work, networking, rendering, or other activities concurrently.

Scenario 2 — Producer–Consumer Data Pipeline

One process/thread produces sensor records and another consumes them. Shared memory can provide a high-speed buffer, while synchronization controls access. Message passing can alternatively provide a structured queue-based communication mechanism.

Scenario 3 — Client–Server Application

A client application communicates with a server using sockets. If the application uses RPC, a client can invoke a remote operation through a higher-level procedure-call abstraction.

Scenario 4 — Multicore Scientific Computation

A large computation is divided into independent tasks. Multiple threads can execute these tasks in parallel on different CPU cores. Amdahl's Law can estimate the maximum benefit based on the fraction that can actually be parallelized.

12. Quick Revision Sheet

Topic	Key point
IPC	Mechanisms that allow processes to communicate and coordinate.
Shared memory	Processes access a common memory region; fast but requires synchronization.
Message passing	Processes exchange messages using send/receive operations.
Direct communication	Processes explicitly identify sender/receiver.
Indirect communication	Messages are exchanged through mailbox/port.
Blocking	Operation may wait until its required condition is satisfied.
Non-blocking	Operation returns/continues without waiting for completion in the same way.
Socket	Communication endpoint used by applications, especially over networks.
RPC	Remote operation presented as a procedure/function call.
Thread	Basic unit of CPU utilization inside a process.
Process	Resource/protection container containing one or more execution contexts.
Concurrency	Multiple tasks make progress during overlapping periods.
Parallelism	Multiple computations execute simultaneously on multiple resources.
Amdahl's Law	Limits overall speedup according to the serial and parallel fractions.

13. Exam-Oriented Questions

Conceptual Questions

- What is IPC? Explain the two major IPC models.
- Compare shared memory and message passing.
- Explain the producer–consumer problem using shared memory.
- Differentiate direct and indirect communication.
- Explain blocking and non-blocking send/receive operations.
- What is a socket? Explain the basic client–server socket communication sequence.
- What is RPC? Explain the role of client and server stubs.
- Define a thread. Explain the relationship between a process and its threads.
- Compare processes and threads.
- Differentiate concurrency and parallelism.

Scenario-Based Questions

- A university system has a producer process generating student records and a consumer process processing them. Which IPC model would you select and why? Explain synchronization concerns.
- A client application needs to request a calculation from a server running on another computer. Explain how sockets and RPC can be used in this system.
- A program contains 25% serial code and 75% parallelizable code. Calculate its theoretical speedup using 4 processors.
- An application becomes unresponsive while waiting for network data. Explain how multithreading could improve responsiveness.

- A system has 8 CPU cores, but only 60% of an application can be parallelized. Explain why adding more cores cannot produce unlimited speedup.

14. Key Takeaways

- IPC is essential whenever independent processes need to exchange information or coordinate.
- Shared memory can be very fast but requires careful synchronization.
- Message passing provides a structured communication mechanism and is especially suitable for isolated or distributed processes.
- Sockets provide communication endpoints; RPC builds a higher-level remote-operation abstraction.
- Threads are execution units within a process and share many process resources.
- Multithreading can improve responsiveness, throughput, and multicore scalability, but synchronization creates complexity.
- Concurrency is about overlapping progress; parallelism is about simultaneous execution.
- Amdahl's Law shows that the non-parallelizable part limits total speedup.

Reference: Silberschatz, Abraham; Galvin, Peter B.; Gagne, Greg. *Operating System Concepts*, Addison-Wesley, 2021. Topics aligned with Chapter 3 (Process Concepts/IPC) and Chapter 4 (Threads & Concurrency).